

Tuning and experiment tracking

Hyperparameters are set before training and chosen by a cross-validated search. A tracker logs every trial so the winner can be found and run again.

THE QUESTION

Which hyperparameter values should you use, and how do you keep track of the many runs it takes to find them?

THE INTUITION

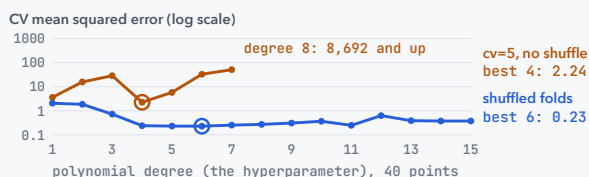
Tuning looks for the peak of an unknown function where each point costs a full cross-validation. Random search tries more values of the knobs that matter; Bayesian optimisation also learns from past scores.

KEY TAKEAWAY

Fix the metric, search 2 to 4 knobs on sensible (often log) ranges with random search or Optuna, log every run, and treat the best score as optimistic.

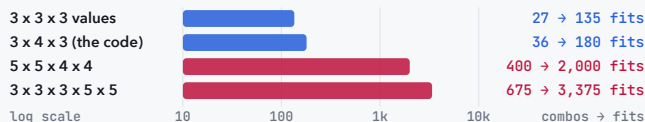
fix the metric → search space → grid · random · Optuna → CV score per trial → log every run → refit, test once

1 Hyperparameters are chosen, parameters are learned



`fit()` learns parameters (weights, split thresholds). You set hyperparameters (`max_depth`, `C`, a polynomial's degree) before it and keep the best CV score. Folds can change the winner: `cv=5` does not shuffle, so on sorted x each fold extrapolates.

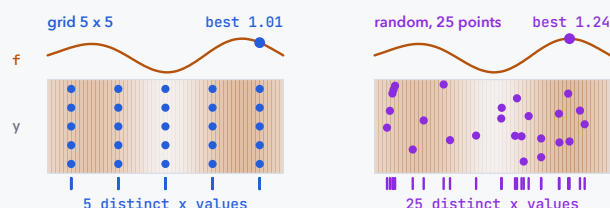
2 Grid search is nested loops, so the cost multiplies



```
grid = GridSearchCV(
    RandomForestClassifier(),
    {"n_estimators": [50, 100, 200],
     "max_depth": [5, 10, 15, None],
     "min_samples_leaf": [1, 2, 5]},
    cv=5, scoring="accuracy", n_jobs=-1)
grid.fit(X_train, y_train)
grid.best_params_, grid.best_score_
```

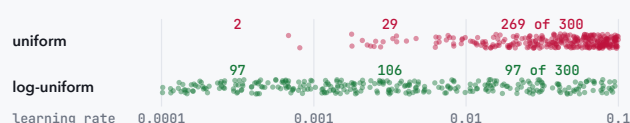
Every extra knob multiplies the fits, so a grid suits 2 or 3 knobs.

3 Random search tries more values of the knob that matters



On a toy score that depends on x only, a 5×5 grid tests 5 values of x and 25 random points test 25 (Bergstra and Bengio, 2012). In 1,000 random draws, random's best beat the grid's 99.3% of the time.

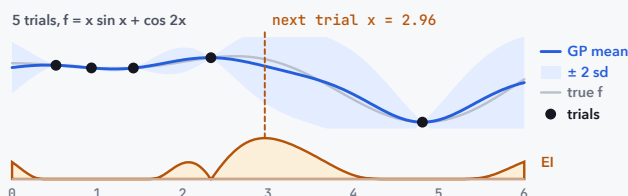
4 Sample from distributions, on a log scale for rates



```
search = RandomizedSearchCV(
    RandomForestClassifier(),
    {"n_estimators": randint(50, 500),
     "max_features": uniform(0.1, 0.9)},
    n_iter=60, cv=5, random_state=42)
```

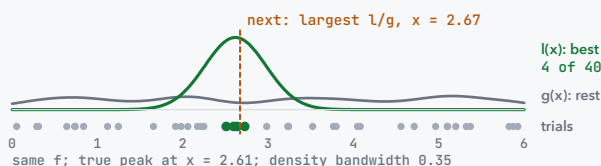
`n_iter` fixes the cost. `randint(50, 500)` stops at 499; `uniform(0.1, 0.9)` means start 0.1, width 0.9. Draw rates and `C` on a log scale with `loguniform`.

5 Bayesian optimisation models the score to pick the next trial



A surrogate with a mean and an uncertainty (often a Gaussian process) is fitted to the trials so far. Expected improvement (EI) is high where the mean is high or the band is wide; the next trial goes to its peak. Here trial 7 scores 1.808 against a true peak of 1.809. Active learning uses the same model but samples where it is least sure.

6 Optuna's default sampler compares good and bad trials

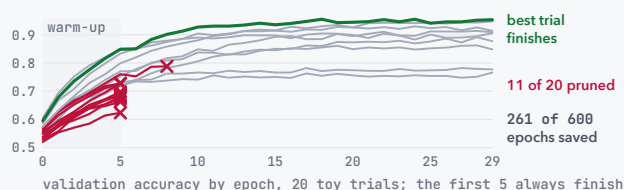


After 10 random trials, TPE splits past trials into the best 10% and the rest, fits a density to each (l and g) and runs the candidate with the largest l/g . It handles integer, log-scale and categorical knobs.

Stop weak trials early and log every run

Prune on validation scores, loop over model families for a ceiling, distrust the best score, keep every run.

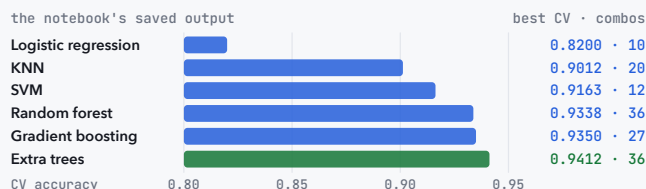
7 Optuna prunes trials that trail the median



```
def objective(trial):
    lr = trial.suggest_float(
        "lr", 1e-4, 1e-1, log=True)
    model = build(lr)
    for epoch in range(30):
        acc = train_and_validate(model)
        trial.report(acc, epoch)
        if trial.should_prune():
            raise optuna.TrialPruned()
    return acc
study = optuna.create_study(
    direction="maximize",
    sampler=TPESampler(seed=42),
    pruner=MedianPruner(n_startup_trials=5,
                        n_warmup_steps=5))
study.optimize(objective, n_trials=100)
```

From step 5 on, a trial stops when its best value so far is below the median of finished trials at that step. Report a validation score; a pruner fed test accuracy tunes on the test set.

8 AutoML loops over model families and their grids



Each family gets its own GridSearchCV; the best CV score wins. AutoGluon, FLAML and auto-sklearn do this at scale. Use it for a ceiling on tabular data, and ship a simpler model if it lands close.

11 Pause and think

5 × 5 × 4 × 4 values, cv=5, 0.5 s a fit? 400 combinations, 2,000 fits, about 17 minutes. One more 5-value knob: 83 minutes.

500 Optuna trials everywhere? Cost grows per trial while the best-so-far curve flattens, and the best score drifts further above the truth.

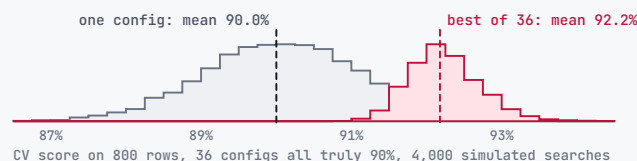
Random 0.9275, grid 0.9338, 36 trials each? The notebook's run. Its five searches sit within 0.7 points, while one config's fold scores run from 0.91 to 0.94.

Scaler fitted before the search? It saw every validation fold. Use a Pipeline.

Learning rates [0.1, 0.2, 0.3]? One decade only. Try [0.001, 0.01, 0.1].

Rerun the 85.7% run? Needs its config, commit, data and seed (the reproducible runs sheet).

9 The best score is a maximum of noisy scores



```
pipe = Pipeline([
    ("scale", StandardScaler()),
    ("model", LogisticRegression())])
search = GridSearchCV(pipe,
    {"model__C": [0.01, 0.1, 1, 10]}, cv=5)
search.fit(X_train, y_train) # refits best
search.score(X_test, y_test) # once
```

Picking the top of 36 equal configs adds 2.2 points of luck to best_score_. Keep preprocessing in the pipeline (step__param names) and score the test set once. Nested CV (the evaluating models sheet) scores the whole search.

10 A tracker keeps config, metrics and artifacts per run

run	config		metrics		metadata
	lr	epochs	val_loss	val_acc	commit
gd-lr0.01-e60	0.01	60	0.635	0.795	3f9c2e1
gd-lr0.1-e60	0.1	60	0.461	0.795	3f9c2e1
gd-lr0.5-e60	0.5	60	0.409	0.790	3f9c2e1
gd-lr0.1-e300	0.1	300	0.409	0.790	3f9c2e1

one row per init ... finish; toy data; commit id illustrative

```
trackio.init(
    project="tuning-demo",
    name="gd-lr0.5-e60",
    config={"lr": 0.5, "epochs": 60})
for epoch in range(60):
    trackio.log({"val_loss": loss,
                "val_acc": acc})
trackio.finish() # then: trackio show
```

Also log artifacts (model file, plots), package versions and tags, and name runs by their settings. W&B uses the same three calls; MLflow has start_run, log_params, log_metric, log_artifact.

metric first · log ranges · random or Optuna · prune on validation · best score runs high · log every run

Source: lecture "Tuning, AutoML and experiment tracking", its tuning and tracking notebook and the hyperparameter tuning lab, nipunbatra.github.io/stt-ai-teaching. Values called the notebook's are quoted from its saved outputs (scikit-learn, Optuna, 1,000 synthetic rows). Figures are numpy computations with fixed seeds: the polynomial data are regenerated with the notebook's code and match its best degree 4 (CV MSE 2.24); the other objectives, curves and runs are toy examples. Bayesian optimisation after Agnihotri and Batra, "Exploring Bayesian optimization", Distill (2020).